

Cloudflare Tunnel로 홈랩 외부 접속하기

Proxmox / ArgoCD를 집 밖에서도 열어보기 — 2026년 8월 정리

노트북을 들고 밖에 나갔을 때도 Proxmox 웹 UI나 ArgoCD에 접속하고 싶어서 시작한 작업이다. Tailscale은 이미 많이 써봤던 터라 이번엔 Cloudflare Tunnel로 해봤다. 두 개는 사실 용도가 다르다. Tailscale은 기기 전체를 하나의 사설망에 묶는 방식이고, Cloudflare Tunnel은 특정 웹 서비스 몇 개를 도메인으로 열어주는 방식에 가깝다. 정확한 용어로는 "site-to-site VPN"이 아니라 "client-to-site" 시나리오였는데, 이 둘을 처음엔 헷갈렸다.

집 네트워크는 이중 NAT 상태다 — 메인 공유기 밑에 ipTIME이 라우터 모드로 물려있어서, 포트포워딩을 하려면 양쪽 라우터를 다 뚫어야 하는 번거로운 구조였다. Cloudflare Tunnel을 선택한 이유도 여기 있다. 포트포워딩이 아예 필요 없는 방식이라 이 문제를 통째로 우회할 수 있었다.

전체 그림

핵심은 `cloudflared` 라는 프로그램이다. 이것을 Proxmox 호스트에 설치해서 돌리면, 이 프로그램이 Cloudflare 쪽으로 먼저 연결을 건다. 서버가 외부에 포트를 여는 게 아니라, 서버가 밖으로 나가는 연결만 유지하는 구조라서, 인바운드 방화벽 규칙이나 포트포워딩이 전혀 필요 없다.

브라우저 ---> Cloudflare 엣지 ---> `cloudflared` (Proxmox 호스트) ---> 내부 서비스
(진짜 인증서) (터널, 아웃바운드 전용) Proxmox :8006 / ArgoCD :30443

설치 순서

실제로 실행했던 명령어들이다. 나중에 다른 서비스를 추가하거나 다른 서버에 똑같이 구성할 때 이 순서 그대로 따라가면 된다.

1 가비아 도메인을 Cloudflare로 연결

Cloudflare 대시보드에서 도메인(`moriental.store`)을 추가하면 네임서버 두 개를 받는다. 이걸 가비아의 도메인 관리 화면에서 기존 네임서버 대신 등록하면, 이후 이 도메인의 DNS는 전부 Cloudflare가 관리하게 된다. 명령어 없이 대시보드에서 하는 작업이라 여기는 스크린샷 기준으로 진행하는 게 낫다.

2 cloudflared 설치 (Proxmox 호스트)

```
sudo mkdir -p --mode=0755 /usr/share/keyrings
curl -fsSL https://pkg.cloudflare.com/cloudflare-main.gpg | sudo tee /usr/share/keyrings/cloudflare-main.gpg >/dev/null

echo 'deb [signed-by=/usr/share/keyrings/cloudflare-main.gpg] https://pkg.cloudflare.com/cloudflared bookworm main' | sudo tee /etc/apt/sources.list.d/cloudflared.list

sudo apt update
sudo apt install -y cloudflared
```

3 터널 인증 + 생성

```
cloudflared tunnel login
cloudflared tunnel create homelab
```

`login` 은 브라우저로 열리는 링크를 통해 Cloudflare 계정과 도메인을 인증하는 단계고, 인증서가 `~/.cloudflared/cert.pem` 에 저장된다. `create` 는 실제 터널을 만들고 터널ID와 자격증명 파일(`<터널ID>.json`)을 생성한다.

4 ingress 설정 작성

```
mkdir -p ~/.cloudflared
cat > ~/.cloudflared/config.yml << 'EOF'
tunnel: 57c7f2e6-e2c8-4eb5-aeae-421d13172889
```

```
credentials-file: /root/.cloudflared/57c7f2e6-e2c8-4eb5-aeae-421d13172889.json

ingress:
  - hostname: proxmox.moriental.store
    service: https://192.168.1.2:8006
    originRequest:
      noTLSVerify: true

  - hostname: argocd.moriental.store
    service: https://192.168.1.101:30443
    originRequest:
      noTLSVerify: true

  - service: http_status:404
EOF
```

호스트네임마다 어느 내부 주소로 보낼지 정의하는 파일이다. 맨 마지막 줄은 위에서 걸리지 않은 나머지 요청을 전부 404로 떨어뜨리는 규칙이라, 실수로 다른 경로가 열리는 걸 막아준다.

5 DNS 라우팅

```
cloudflared tunnel route dns homelab proxmox.moriental.store
cloudflared tunnel route dns homelab argocd.moriental.store
```

호스트네임마다 한 번씩 실행해서 CNAME 레코드를 만든다. config.yml에 hostname을 적어두는 것만으로는 DNS 레코드가 자동으로 생기지 않으니, 이 명령어를 빼먹으면 접속 시 NXDOMAIN이 뜬다.

6 Cloudflare Access 정책 설정

Zero Trust 대시보드 → Access controls → Applications에서 Self-hosted / Public Hostname 애플리케이션을 호스트네임마다 하나씩 만들고, Policy에서 `Include: Emails`로 허용할 이메일 주소를 지정한다. 여기까지 안 하면 터널은 열려 있어도 인증 없이 전 세계 누구나 들어올 수 있는 상태가 된다.

7 상시 서비스 등록

```
sudo cloudflared service install
sudo systemctl status cloudflared
```

이 명령어 전까지는 `cloudflared tunnel run` 을 터미널에서 직접 실행해야만 터널이 유지되는 상태였다. `service install` 은 systemd 서비스로 등록해서, SSH 세션이 끊기거나 재부팅해도 터널이 계속 살아있게 만든다.

8 검증

```
# DNS가 실제로 잡히는지 확인 (8.8.8.8은 캐싱 때문에 늦게 반영될 수 있어 1.1.1.1로 확인)
nslookup proxmox.moriental.store 1.1.1.1

# cloudflared 자체 연결 상태 확인
sudo journalctl -u cloudflared -n 50 --no-pager
```

그 다음 브라우저로 `https://proxmox.moriental.store` ,
`https://argocd.moriental.store` 에 각각 접속해서 Access 로그인 화면이 뜨고, 인증 후 실제 서비스 화면까지 도달하는지 확인하면 끝이다.

개념 정리

HTTPS는 사실 세 구간으로 나뉜다

Proxmox와 ArgoCD는 둘 다 self-signed 인증서를 쓰고 있어서 브라우저에서 직접 열면 경고가 뜬다. 이걸 cert-manager 같은 걸로 고쳐야 하나 싶었는데, 실제로는 그럴 필요가 없었다. 위 그림의 화살표 두 개가 각각 다른 구간이기 때문이다.

브라우저와 Cloudflare 엣지 사이는 Cloudflare가 도메인을 추가하는 순간 자동으로 발급해주는 무료 인증서(Universal SSL)로 처리된다. 사용자가 보는 자물쇠 표시는 이 구간의 것이다. Cloudflare 엣지와 cloudflared 사이는 터널 프로토콜 자체가 암호화되어 있다. 그리고 cloudflared와 실제 서비스 사이, 여기가 self-signed 인증서가 있는 구간인데, 사실 LAN 안에서만 오가는 연결이라 검증을 굳이 강제할 필요가 없다. 그래서 config.yml의 `noTLSVerify: true` 가 이 구간의 검증을 건너뛰라는 뜻이다.

ingress라는 단어가 두 번 나온다

쿠버네티스의 `Ingress` 리소스와 cloudflared 설정 파일의 `ingress:` 항목은 이름만 같고 완전히 다른 개념이다. k8s의 Ingress는 클러스터 안에서 도는 서비스들한테 트래픽을 나눠주는 오브젝트고, 클러스터 안에 Ingress Controller가 떠 있어야 동작한다.

cloudflared의 `ingress:` 는 이 프로그램 혼자 들고 있는 라우팅 표일 뿐이다. "이 호스트네임으로 오면 이 IP:포트로 보내라"는 규칙을 나열해둔 것뿐이고, k8s가 깔려 있는지 아닌지는 전혀 상관하지 않는다. 실제로 config.yml의 첫 줄(Proxmox)은 k8s와 무관한 대상을 가리키고, 둘째 줄(ArgoCD)만 우연히 k8s NodePort를 가리킨다.

DNS는 CNAME으로 연결된다

`tunnel route dns` 가 만드는 건 A 레코드가 아니라 CNAME이다. 특정 IP 주소 하나에 고정하는 대신, `<터널 ID>.cfargotunnel.com` 이라는 Cloudflare가 관리하는 호스트네임을 가리키게 해두는 방식이다. Cloudflare 쪽 옛지 서버 IP가 바뀌어도 이쪽 DNS 레코드는 그대로 둘 수 있다는 게 이 방식의 이유다.

Tunnel과 Access는 서로 다른 문이다

Tunnel을 만들고 DNS까지 연결하면 통로는 열리지만, 이 통로 자체에는 인증 기능이 없다. 호스트네임만 알면 전 세계 누구나 들어올 수 있는 상태가 된다는 뜻이다. 그래서 별도로 Cloudflare Access를 앞에 세워서, 지정한 이메일만 통과시키도록 정책을 걸었다.

정책은 `Include: Emails = your-email@example.com` 하나뿐이다. 그런데 실제로 테스트해보니 이메일로 오는 인증코드를 입력하는 화면이 안 뜨고 바로 통과되는 경우가 있었다. 원인은 Zero Trust의 로그인 방법(Login Methods)에 "Cloudflare 계정으로 로그인"이 켜져 있었고, 브라우저에 이미 그 계정으로 로그인된 상태였기 때문이다. 이메일 인증코드와 Cloudflare 계정 로그인은 둘 다 "이 사람이 정말 그 이메일의 주인이다"를 증명하는 방법일 뿐이고, 정책은 어느 방법으로 증명하든 상관하지 않는다. 실제로 다른 이메일로 로그인을 시도하면 정책에서 그대로 걸려진다 — 확인해본 결과 보안 구멍은 아니었다.

트러블슈팅 노트

설정 자체는 다 끝났는데 접속이 안 되는 구간이 두 번 있었다. 둘 다 원인은 DNS 응답이 어딘가에 캐싱되어 있었던 것뿐, 실제 설정 문제는 아니었다.

첫 번째는 `nslookup` 으로 조회했을 때 분명히 레코드를 만들었는데도 계속 `NXDOMAIN` 이 나온 경우였다. 8.8.8.8(구글 공개 DNS)이 레코드가 생기기 전의 "없음" 응답을 잠깐 기억하고 있었던 것이었고, `1.1.1.1` 로 직접 조회하니 바로 정상적으로 잡혔다.

두 번째는 OS 차원의 DNS 캐시(`ipconfig /flushdns`)를 지웠는데도 브라우저에서만 계속 `DNS_PROBE_FINISHED_NXDOMAIN` 이 뜬 경우였다. Chrome과 Edge는 시스템 DNS 캐시와 별개로 자체 Secure DNS(DNS-over-HTTPS) 캐시를 갖고 있어서, 시스템 DNS 서버를 아예 `1.1.1.1`로 바꾸고 나서야 풀렸다.

그 외에 겪었던 자잘한 것들: Cloudflare 온보딩 화면이 "Add a Site"에서 "Add a domain"으로 이름이 바뀌어 있었고, 완전히 빈 도메인은 레코드가 하나도 없으면 활성화가 안 돼서 더미 A 레코드(`@ → 192.0.2.1`)를 하나 넣어줘야 했다. "원본 서버에 Cloudflare IP만 허용하라"는 안내도 뒀는데, 이건 원본이 공인 IP로 직접 노출된 전통적 구성을 위한 안내라 Tunnel 방식에는 해당하지 않는다 — 애초에 인바운드 포트가 없으니 원본 직접 접근 자체가 불가능하다.

실제 설정값

터널 이름	homelab
터널 ID	57c7f2e6-e2c8-4eb5-aeae-421d13172889
도메인	moriental.store (가비아에서 구매, 네임서버를 Cloudflare로 위임)
proxmox.moriental.store	https://192.168.1.2:8006 (Proxmox 웹 UI)
argocd.moriental.store	https://192.168.1.101:30443 (ArgoCD, NodePort)
Access 허용 이메일	your-email@example.com
cloudflared 실행 위치	Proxmox 호스트, systemd 서비스로 상시 실행

설정 파일

~/cloudflared/config.yml

나중에 서비스를 하나 더 열고 싶으면, config.yml에 hostname/service 한 쌍을 추가하고
`cloudflared tunnel route dns homelab <새 호스트네임>` 을 한 번 더 실행한 뒤
Access에 애플리케이션을 하나 더 만들면 된다.